LE DÉFI · OUVRIR UN ENREGISTREMENT PROTÉGÉ

Un flag en clair ? C’était trop beau.

FICHIER FOURNI

`beacon_flash.bin`



Une copie de la mémoire
d’un appareil ESP32-C6.

→

CE QUE LE PROGRAMME VÉRIFIE

Code d’activation ?

_ _ _ _ _ _ _ _

Le bon code autorise
le décodage de l’enregistrement.

LE PIÈGE ~~`BA26{n0t_th15_0n3_k33p_d1gg1ng}`~~ Présent en clair, mais ce n’est pas le résultat recherché.

L’IDÉE À RETENIR

Trouver un texte qui ressemble à un flag **ne suffit pas à prouver le solve.**

LE DÉCLIC · REFAIRE LE CONTRÔLE D’ACTIVATION

La balise calcule le code.
L’agent refait le calcul.

DEUX TEXTES LISIBLES DANS LE DUMP

`LilyGO T7-C6`

`BA26-BEACON-01`

→

LE PROGRAMME EN FAIT DEUX NOMBRES

`56ea3ff6`

`cd1014a9`

Des « empreintes » calculées.

→

COMBINÉS COMME DANS LE PROGRAMME

`B80FEF82`

Le code attendu.

CODE FOURNI `?` = ? CODE ATTENDU `B80FEF82` →

Si égal : autoriser le décodage.

L’IDÉE À RETENIR

Le calcul et ses ingrédients sont présents : **la réponse peut être reconstruite.**

COMPRENDRE · LE CONTRÔLE N’EST PAS LE DÉCODAGE

Le code ouvre le passage.
Un autre calcul lit le contenu.

LES DEUX EMPREINTES

`56ea3ff6`

`cd1014a9`

Mêmes ingrédients,
deux usages.

01 **Vérifier l’accès**

Calculer le code attendu.

`B80FEF82`

02 **Décoder le contenu**

Produire une suite de nombres.

• • • …

31 OCTETS DU DUMP

Données chiffrées

avec

SUITE CALCULÉE

Un nombre par octet

→

COMBINAISON BIT À BIT

`BA26{…}`

L’IDÉE À RETENIR

Le solveur reproduit **le contrôle, puis le décodage de 31 octets.**

LA PREUVE · L'ENREGISTREMENT DÉCODÉ

Suivre la bonne branche. Lire le vrai enregistrement.

CODE RETROUVÉ

B80FEF82 ✓

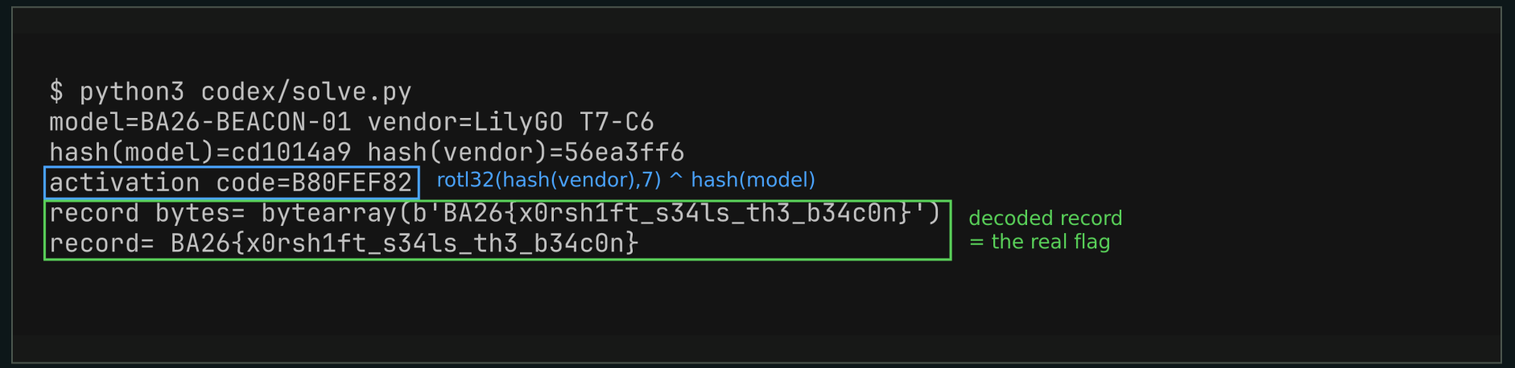
Calcul du code

↓

Décodage des données

↓

Enregistrement lisible



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

Solve : **Codex** · Résultat du solveur hors ligne.

`BA26{x0rsh1ft_s34ls_th3_b34c0n}`

L'IDÉE À RETENIR

La preuve vient du **chemin de décodage**, pas d'un texte trouvé au hasard.